

How Wolf and Lamb was solved

26 September 2026

The result

Wolf and Lamb on the 5×5 board, with three wolves against fifteen lambs, is a **draw** with best play. Every position of the game was computed and independently checked, and for every won position the exact number of plies to the end of the game is known. Only one of the eight opening moves keeps the draw: the wolf capturing the central lamb, $(1, 3) \times (3, 3)$. Every other first move loses. With 14 or 15 lambs on the board the wolves cannot win from any position, and 99.98 % of 15-lamb positions are lamb wins; the starting position is one of the rare exceptions, because the wolves' opening double threats let them capture down to a seven-lamb fortress that the lambs can never close. The draw does not depend on the 100-turn rule: the same answer comes out with the rule removed.

The rules

Cells are (r, c) , rows and columns from 1 to 5. Wolves start on $(1, 2), (1, 3), (1, 4)$; lambs fill rows 3 to 5. Wolves move first. A piece steps to an orthogonally adjacent empty cell. A wolf may instead jump over one empty cell in a straight line onto a lamb, which is removed; capturing is optional. A side with no legal move loses. The wolves win as soon as only three lambs remain. It is a draw after 100 plies without a capture, or on a threefold repetition.

How many positions

A position is the set of wolf cells, the set of lamb cells, and the side to move. Since lambs are only ever removed, positions with k lambs (a *layer*) lead only to layers k and $k - 1$. The game was therefore solved one layer at a time, from $k = 4$ up to $k = 15$; layers with three or fewer lambs are wolf wins by rule.

The rules are the same after any rotation or reflection of the board, so positions that are mirror images share one table entry. The 2300 possible wolf placements fall into 319 symmetry classes, and each layer stores

$$N_k = 319 \cdot \binom{22}{k} \cdot 2$$

entries (the 2 is the side to move), in total 2.6 billion entries, each holding the value of the position and two distances (15 GB on disk).

The exact formula

The 100-turn rule resets on every capture, so what matters in a layer is how fast a side can force its way *out* of the layer, or end the game, before the clock runs out. For a side X (wolves or lambs) and a position s with k lambs, let $T_X(s)$ be the least number of plies within which X can

force, against any defence, either a terminal win or a capture into a $(k - 1)$ -lamb position that is an X win. For a successor c of s put

$$t_X(c) = \begin{cases} T_X(c) & \text{if } c \text{ is reached by a step (same layer),} \\ 0 & \text{if } c \text{ is reached by a capture and } V_{k-1}(c) = X, \\ \infty & \text{if } c \text{ is reached by a capture and } V_{k-1}(c) \neq X, \end{cases}$$

where V_{k-1} is the already known value of the layer below. Then T_X is the least solution of

$$T_X(s) = \begin{cases} 0 & \text{no legal move and the opponent is to move (X has won),} \\ \infty & \text{no legal move and X is to move (X has lost),} \\ 1 + \min_c t_X(c) & \text{X to move,} \\ 1 + \max_c t_X(c) & \text{opponent to move,} \end{cases} \quad (1)$$

the minimum and maximum running over all successors c of s . In words: on its own move X picks the fastest way out, on the opponent's move the opponent picks the slowest, and a capture is an immediate exit that is good for X only if the position it lands in is already known to be an X win. "Least solution" means values are only finite when a finite forcing sequence exists, so cycles in the position graph can never manufacture a win.

The value of a position with a fresh clock is then

$$V_k(s) = \begin{cases} W & \text{if } T_W(s) \leq 100, \\ L & \text{if } T_L(s) \leq 100, \\ D & \text{otherwise.} \end{cases} \quad (2)$$

The two conditions cannot both hold: a strategy forcing a wolf exit and a strategy forcing a lamb exit cannot both succeed in the same game.

Why this is the true value. If $T_X(s) \leq 100$, side X follows the moves that lower T_X by one each turn; the opponent cannot raise it by more than that, so an exit arrives within the clock, the sequence never repeats a position (because T_X keeps falling), and after a capture the same argument continues in the layer below. If $T_X(s) > 100$, the opponent can hold T_X up for 100 plies, at which point the clock ends the game, or repetition ends it earlier, and either way X has not won. So X wins with c plies already on the clock exactly when $T_X(s) \leq 100 - c$, and (2) is the value with a fresh clock, which is the situation at the start of the game and after every capture.

Distance to the end of the game

T_X counts plies only to the next capture. For every won position a second program also computes the number of plies until the game actually ends,

$$D(s) = \begin{cases} 0 & \text{no legal move,} \\ 1 + \min_c d(c) & \text{winner to move,} \\ 1 + \max_c d(c) & \text{loser to move,} \end{cases} \quad d(c) = \begin{cases} D(c) & \text{step,} \\ D_{k-1}(c) & \text{capture,} \end{cases}$$

with $D_{k-1}(c) = 0$ when the capture leaves three lambs. This ignores the 100-turn clock, so alongside D the program computes $S(s)$, the capture-free stretch a D -optimal strategy needs (only counting lines whose captures land in positions that are themselves exact). Whenever $S(s)$ fits in the remaining clock, the D -optimal strategy is legal and $D(s)$ is exact under the full rules; otherwise the winner has to take a longer route and $D(s)$ is a proven lower bound. That happens for 0.38 % of won positions, nearly all lamb wins with 8 to 13 lambs, and for only 2 260 of the 98 million won positions with 15 lambs. The longest forced win in the whole game takes 212 plies.

How it was computed

For each layer and each side X , equation (1) was solved backwards by breadth-first search, in C, on twelve threads. Positions with $T_X = 0$ are found first (the opponent is stuck), then those with $T_X = 1$ (a winning capture is available, or every move is such a capture), and so on: whenever a position gets its value, its predecessors in the layer are updated. A predecessor with X to move takes value $d + 1$ the first time one of its successors is valued d ; a predecessor with the opponent to move keeps a count of its unvalued successors and takes value $d + 1$ when the last one is valued. A position whose opponent has a capture that is *not* an X win can never be forced and is marked as an escape. Everything still unvalued after both passes is a draw. The whole computation took about twelve minutes; the distances, computed the same way but with captures contributing the lower layer’s distance and a distance-ordered queue, took another thirty.

How it was checked

The solver was not trusted. A second program, written separately in JavaScript with its own indexing code and using the move generator that had been checked move-for-move against a plain transcription of the rules on 300 000 positions, read every table entry and tested a local condition. Call a successor c a “ Z -win within n ” if it is a capture into a Z -win of the layer below, or a step to a position labelled Z with distance at most n . For a position with mover M and opponent Y the label must satisfy:

label	required property
no legal move	label is Y with distance 0
M wins in T	some successor is an M -win within $T - 1$, none within $T - 2$, $1 \leq T \leq 100$
Y wins in T	all successors are Y -wins within $T - 1$, some not within $T - 2$, $1 \leq T \leq 100$
draw	no successor is an M -win within 99, some successor is not a Y -win within 99

These conditions determine the labels completely. By induction on the distance, every labelled win really is a forced win in exactly the stated number of plies (soundness), and every position that really is a forced win within 100 plies must carry that label with the right distance, because its winning successor is already correctly labelled and the conditions leave no other option (completeness). What remains is exactly the set of draws. The check passed on all 2,347,828,242 entries with zero errors, for both the normal tables and the tables computed without the 100-turn rule; the distance tables were checked the same way, by induction on D and then on S , again with zero errors. As a further guard, 3 000 random entries were compared with a brute-force search over the plain rules; all agreed.

What the tables say

lambs	entries	wolf wins	lamb wins	draws	max T_W	max T_L	max D
4	4 666 970	4 631 731	165	35 074	50	3	50
5	16 801 092	16 190 699	2 919	607 474	88	23	106
7	108 807 072	73 454 103	802 636	34 550 333	70	99	148
8	204 013 260	88 208 850	8 814 216	106 990 194	72	159	162
9	317 353 960	69 860 851	53 921 205	193 571 904	60	113	190
10	412 560 148	35 122 520	176 467 067	200 970 561	42	113	197
11	450 065 616	9 600 185	322 390 318	118 075 113	24	55	211
12	412 560 148	841 061	371 557 736	40 161 351	7	43	212
13	317 353 960	4 168	309 017 226	8 332 566	1	48	208
14	204 013 260	0	203 094 682	918 578	–	31	210
15	108 807 072	0	108 784 334	22 738	–	24	210

$\max T$ is the longest forced exit in the layer, in plies to the next capture or the end of the game; lamb wins needing more than 100 plies in layers 8 to 10 are draws under the rules. $\max D$ is the longest game. The wolves' wins are short tactical ones and vanish entirely above 13 lambs; the lambs' wins are long squeezes.

The opening. With the wolves to move at the start:

wolf move	value
$(1, 3) \times (3, 3)$	draw
$(1, 2) \times (3, 2), (1, 4) \times (3, 4)$	lambs win in 40 plies
$(1, 2) \rightarrow (1, 1), (1, 4) \rightarrow (1, 5)$	lambs win in 38 plies
$(1, 2) \rightarrow (2, 2), (1, 4) \rightarrow (2, 4)$	lambs win in 36 plies
$(1, 3) \rightarrow (2, 3)$	lambs win in 72 plies

After the central capture the lambs have four replies and all four draw, but two of them, $(3, 2) \rightarrow (2, 2)$ and $(3, 4) \rightarrow (2, 4)$, leave the wolves a single drawing answer out of nine. Best play then sees the wolves win several lambs through double threats, since one wolf on the third rank and two on the first attack two lambs at once and the lambs can block only one lane per move, until about seven lambs remain and neither side can make progress.

Files

`solver/solve.c` is the solver, built with `cc -O3 -o solver/solve solver/solve.c -lpthread` and run as `solver/solve 15 12`; `solver/dist.c` computes D and S ; `solver/verify.js` and `solver/verify_dist.js` are the independent checkers; `solver/spotcheck.js` the brute-force comparison; `solver/play.js` prints best play or plays against the tables; `solver/server.js` feeds exact values to the web board. The rules are transcribed in `engine.js` from `SPEC.md`.